

EXP-3

Handling missing data

Name-Shafiya Bennet Singh

Reg no-25BAI0199

```
import pandas as pd
import numpy as np
```

```
# Let's assume we have a dataframe with names and ages
```

```
data = {
    "name": ["Alice", "Bob", "Charlie", "David", "Emma"],
    "age": [21, 22, 20, 23, 24]
}
```

```
dfx = pd.DataFrame(data, index=["id1", "id2", "id3", "id4", "id5"])
print(dfx)
```

```
# Let's add some missing values
```

```
dfx["city"] = np.nan
dfx.loc["id6"] = ["Frank", 25, "Chennai"]
dfx.loc["id7"] = np.nan
dfx["phone"] = np.nan
```

```
dfx.loc["id1", "city"] = "Mumbai"
dfx.loc["id3", "city"] = "Delhi"
```

```
print(dfx)
```

```
# Check for missing values
```

```
dfx.isnull()
```

```
# Check non-missing values
```

```
dfx.notnull()
```

```
# Count missing values in each column
```

```
dfx.isnull().sum()
```

```
# Total number of missing values
```

```
dfx.isnull().sum().sum()
```

```
# Count non-missing values
```

```
dfx.count()
```

```
# Display only non-null values from city column
```

```
dfx.city[dfx.city.notnull()]
```

```
# Drop missing values from city column
```

```
dfx.city.dropna()
```

```
# Drop rows containing any NaN values
```

```
dfx.dropna()
```

```
# Drop rows where all values are NaN
```

```
dfx.dropna(how="all")
```

```
# Drop columns where all values are NaN
```

```
dfx.dropna(how="all", axis=1)
```

```
# Keep columns with at least 5 non-null values
```

```
dfx.dropna(thresh=5, axis=1)
```

```
# Mathematical operations with NaN
```

```
ar1 = np.array([21, 22, np.nan, 24])
```

```
ser1 = pd.Series(ar1)
```

```
print(ar1.mean(), ser1.mean())
```

```
# Sum of ages
```

```
ser2 = dfx["age"]
```

```
print(ser2.sum())
```

```
# Average age
```

```
print(ser2.mean())
```

```
# Cumulative sum of ages
```

```
print(ser2.cumsum())
```

```
# Fill missing values with 0
```

```
filledDf = dfx.fillna(0)
```

```
print(filledDf)
```

```
# Difference in mean before and after filling
```

```
print(dfx.mean(numeric_only=True))
```

```
print(filledDf.mean(numeric_only=True))
```

```
# Forward filling
```

```
print(dfx["city"].fillna(method="ffill"))
```

```
# Backward filling
```

```
print(dfx["city"].fillna(method="bfill"))
```